

Security Assessment — Acme Corp

Engagement: Phase-1 assessment · Date: 2026-07-15 · Overall severity: Critical

Critical: 2 · High: 3

AI-assisted draft. Must be reviewed and signed off by a human analyst before release. Reviewed by: _____ Date: _____

Executive summary

Acme Corp's Phase-1 assessment of test.aiforyourwork.net identified critical SQL injection vulnerabilities in the WebGoat training environment and significant exposure of non-production infrastructure. Dynamic testing corroborated two live SQL injection endpoints enabling unauthorized database queries, while static analysis revealed three high-exploitability SQL injection patterns in the codebase where user input flows unsanitized into JDBC statements. Additionally, subdomain enumeration exposed 14 high-risk non-prod environments (dev, staging, QA, admin, SSO, API v2) and monitoring/webhook endpoints accessible at HTTP 401, creating a layered attack surface. Immediate remediation must focus on eliminating SQL injection via parameterized queries across all identified locations, restricting non-prod environment exposure, and segmenting development infrastructure from production networks.

Findings

#	Severity	Conf.	Finding	Asset	CWE / OWASP / ATT&CK
1	Critical	High	SQL Injection in JDBC Statement Execution (WebGoat Lesson Assignment 5b & Attack 10)	http://localhost:8080/WebGoat/SqlInjection/assignment5b (parameter: userid) and http://localhost:8080/WebGoat/SqlInjection/attack10 (parameter: action_string)	CWE-89 · A03:2021 – Injection · T1190
2	Critical	High	SQL Injection via Unsanitized JWT 'kid' Header in Token Processing	src/main/java/org/owasp/webgoat/lessons/jwt/claimmisuse/JWTHeaderKIDEndpoint.java:73 (line 76 SQL query)	CWE-89 · A03:2021 – Injection · T1190
3	High	High	Non-Production Environments Exposed via Public Subdomain Enumeration	api-v2.test.aiforyourwork.net, staging.test.aiforyourwork.net, dev.test.aiforyourwork.net, admin.test.aiforyourwork.net, qa.test.aiforyourwork.net, webhook.test.aiforyourwork.net, internal.test.aiforyourwork.net, billing.test.aiforyourwork.net (HTTP 401 responses revealing service purpose)	CWE-200 · A01:2021 – Broken Access Control · T1526
4	High	High	SSO and Admin Console Exposure via Public HTTP 200 Landing Page	sso.test.aiforyourwork.net (HTTP 200 'Sign in – PayBee SSO'), admin.test.aiforyourwork.net (HTTP 401 'Sign in – PayBee Admin')	CWE-200 · A01:2021 – Broken Access Control ·

					T1190, T1110
5	High	High	Unprotected API and Monitoring Endpoints (HTTP 200)	api.test.aiforyourwork.net (HTTP 200 'PayBee API Portal'), payments.test.aiforyourwork.net (HTTP 200 'PayBee Payments'), monitoring.test.aiforyourwork.net (HTTP 200 'PayBee Monitoring'), partner-portal.test.aiforyourwork.net (HTTP 200 'PayBee Partner Portal')	CWE-200 · A01:2021 – Broken Access Control · T1526, T1592

F1. SQL Injection in JDBC Statement Execution (WebGoat Lesson Assignment 5b & Attack 10)

Critical

confidence High · Static + Dynamic (corroborated)

Asset: http://localhost:8080/WebGoat/SqlInjection/assignment5b (parameter: userid) and http://localhost:8080/WebGoat/SqlInjection/attack10 (parameter: action_string)

Classification: CWE-89 · A03:2021 – Injection · T1190

Evidence: Dynamic testing (sqlmap) confirmed SQL injection on two WebGoat endpoints: (1) /assignment5b parameter 'userid' vulnerable to time-based blind and UNION query techniques exploiting HSQLDB; (2) /attack10 parameter 'action_string' vulnerable to boolean-based blind and UNION query with 3-column DBMS. Both confirmed to enable arbitrary database query execution. Static analysis (semgrep) identified corresponding high-exploitability patterns in LessonConnectionInvocationHandler.java:31 and UserService.java:53 where username and user input flow unsanitized into JDBC statement.execute() and CREATE SCHEMA queries via String concatenation without parameterization.

Remediation: Replace all instances of String concatenation in SQL query construction with parameterized queries using PreparedStatement with bound parameters. Specifically: (1) In LessonConnectionInvocationHandler.java:31, refactor to use PreparedStatement with '?' placeholders for username values; (2) In UserService.java:53, replace concatenated CREATE SCHEMA statement with parameterized equivalent (e.g., using CallableStatement if HSQLDB supports DDL parameterization, or validate schema names against a whitelist and escape appropriately); (3) Validate and test both endpoints to confirm injection vectors are eliminated. Apply same pattern across all JDBC query construction throughout codebase.

F2. SQL Injection via Unsanitized JWT 'kid' Header in Token Processing **Critical**

confidence High · Static (semgrep)

Asset:

src/main/java/org/owasp/webgoat/lessons/jwt/claimmisuse/JWTHeaderKIDEndpoint.java:73 (line 76 SQL query)

Classification: CWE-89 · A03:2021 – Injection · T1190

Evidence: Semgrep analysis identified high-exploitability SQL injection pattern at JWTHeaderKIDEndpoint.java line 76 where JWT header 'kid' parameter flows directly into SQL query via String concatenation: SELECT key FROM jwt_keys WHERE id = "" + kid + ". Attacker controls 'kid' value through malicious JWT token. Exploit path: craft JWT with malicious 'kid' header (e.g., "" OR '1'='1") to execute arbitrary SQL against jwt_keys table and extract sensitive key material.

Remediation: Refactor the query to use parameterized PreparedStatement with bound parameters for the 'kid' value. Example: PreparedStatement pstmt = connection.prepareStatement("SELECT key FROM jwt_keys WHERE id = ?"); pstmt.setString(1, kid);. Additionally, validate and sanitize JWT 'kid' header values against a whitelist of expected key identifiers before database query execution. Log and reject any JWT with unexpected 'kid' values.

F3. Non-Production Environments Exposed via Public Subdomain Enumeration High

confidence High · Recon

Asset:

api-v2.test.aiforyourwork.net, staging.test.aiforyourwork.net, dev.test.aiforyourwork.net, admin.test.aiforyourwork.net, qa.test.aiforyourwork.net, webhook.test.aiforyourwork.net, internal.test.aiforyourwork.net, billing.test.aiforyourwork.net (HTTP 401 responses revealing service purpose)

Classification: CWE-200 · A01:2021 – Broken Access Control · T1526

Evidence: Subdomain enumeration against test.aiforyourwork.net identified 14 high-risk non-prod hosts with HTTP 401 responses: api-v2.test.aiforyourwork.net, staging.test.aiforyourwork.net, dev.test.aiforyourwork.net, admin.test.aiforyourwork.net, qa.test.aiforyourwork.net, webhook.test.aiforyourwork.net, internal.test.aiforyourwork.net (explicitly labeled non-prod), and billing.test.aiforyourwork.net. Banner grabbing reveals service purpose (e.g., 'PayBee API v2 (development)', 'PayBee Dev', 'PayBee Admin'). HTTP 401 indicates authentication exists but infrastructure architecture is disclosed, aiding adversary reconnaissance.

Remediation: Restrict non-production environment DNS registration and subdomain visibility: (1) Do not publish non-prod subdomains in public DNS; isolate to internal DNS or VPN-only access; (2) Move non-prod environments off public-facing test.aiforyourwork.net domain onto internal-only namespace or private IP ranges; (3) If public access is required for testing, implement IP whitelisting and require multi-factor authentication at the perimeter; (4) Disable banner grabbing and service identification responses on all non-prod endpoints (remove application banner strings from HTTP headers and HTML titles). (5) Segment dev/staging/QA network traffic from production to limit lateral movement if a non-prod environment is compromised.

F4. SSO and Admin Console Exposure via Public HTTP 200 Landing Page High

confidence High · Recon

Asset:

sso.test.aiforyourwork.net (HTTP 200 'Sign in – PayBee SSO'), admin.test.aiforyourwork.net (HTTP 401 'Sign in – PayBee Admin')

Classification: CWE-200 · A01:2021 – Broken Access Control · T1190, T1110

Evidence: Subdomain enumeration discovered sso.test.aiforyourwork.net responding with HTTP 200 and login page banner 'Sign in — PayBee SSO' and admin.test.aiforyourwork.net responding with HTTP 401 and 'Sign in — PayBee Admin' banner. Both are publicly reachable identity and administrative entry points, enabling credential enumeration, brute-force attack staging, and phishing against admin/privileged users.

Remediation: Restrict SSO and admin console access to authenticated corporate networks or VPN: (1) Move sso.test.aiforyourwork.net and admin.test.aiforyourwork.net off public DNS or implement IP-based access control (restrict to corporate IP ranges only); (2) Implement rate limiting and account lockout policies on both login forms to defeat brute-force attacks; (3) Enable multi-factor authentication (MFA) for all admin and SSO accounts; (4) Monitor and log all login attempts with alerting for failed attempts and anomalous access patterns. (5) Implement CAPTCHA or challenge-response mechanisms to prevent automated credential stuffing.

F5. Unprotected API and Monitoring Endpoints (HTTP 200) High

confidence High · Recon

Asset:

api.test.aiforyourwork.net (HTTP 200 'PayBee API Portal'), payments.test.aiforyourwork.net (HTTP 200 'PayBee Payments'), monitoring.test.aiforyourwork.net (HTTP 200 'PayBee Monitoring'), partner-portal.test.aiforyourwork.net (HTTP 200 'PayBee Partner Portal')

Classification: CWE-200 · A01:2021 – Broken Access Control · T1526, T1592

Evidence: Subdomain enumeration identified four HTTP 200 endpoints with no apparent authentication challenge at the landing page: `api.test.aiforyourwork.net` (API Portal), `payments.test.aiforyourwork.net` (financial service), `monitoring.test.aiforyourwork.net` (operational metrics tool), and `partner-portal.test.aiforyourwork.net` (partner authentication portal). These are reachable without credentials at first access, enabling information disclosure about API structure, authentication mechanisms, and operational infrastructure.

Remediation: Implement access control and authentication on all sensitive endpoints: (1) API Portal (`api.test.aiforyourwork.net`): Require API key authentication and OAuth/JWT tokens for all endpoints; document API schema only after authentication; (2) Payments service (`payments.test.aiforyourwork.net`): Require user authentication before any financial data access; implement PCI-DSS compliant access controls; (3) Monitoring (`monitoring.test.aiforyourwork.net`): Restrict to authenticated internal staff only; move off public domain or enforce IP whitelisting and VPN requirement; (4) Partner Portal: Implement OAuth/SAML with partner identity provider; require multi-factor authentication. For all endpoints, validate authentication on every request, not just landing page.

Attack surface

The `test.aiforyourwork.net` domain exposes a broad non-production infrastructure footprint with 31 live subdomains. Fourteen hosts return HTTP 200 or HTTP 401 responses indicating active services, including: customer-facing portals (API v1/v2, payments, partner portal), identity infrastructure (SSO login), administrative consoles (admin, internal tools), and development/testing environments (dev, staging, QA, webhook, billing). The HTTP 401 responses indicate authentication exists but reveal service purpose and architecture through banner grabbing, aiding reconnaissance. Monitoring and webhook endpoints are reachable, potentially exposing operational metrics or integration logic. This enumeration footprint—spanning development, staging, QA, and admin tiers—substantially increases reconnaissance efficiency for an attacker and widens the perimeter an organization must defend.

Recommendations

1. ****IMMEDIATE (Week 1):**** Remediate SQL injection vulnerabilities by refactoring all identified JDBC query construction to use parameterized PreparedStatements. Prioritize the three high-exploitability patterns in `LessonConnectionInvocationHandler.java:31`, `UserService.java:53`, and `JWTHeaderKIDEndpoint.java:76`. Validate fixes via both static code review and dynamic re-testing with `sqlmap`.
2. ****IMMEDIATE (Week 1–2):**** Restrict public DNS exposure of non-production environments. Migrate dev, staging, QA, admin, and internal tool subdomains to internal-only DNS or VPN-protected access. If public access is unavoidable for testing, implement IP whitelisting, disable service banner grabbing, and require multi-factor authentication at the perimeter.
3. ****HIGH PRIORITY (Week 2–3):**** Isolate non-production infrastructure from production networks via network segmentation (separate VLANs, firewall rules). Document and enforce access control policies distinguishing dev, staging, QA, and production tiers. Apply principle of least privilege to all environment access.
4. ****HIGH PRIORITY (Week 2–3):**** Harden authentication on all sensitive endpoints (SSO, admin console, API portals, payments, monitoring). Enforce multi-factor authentication for all admin and privileged user accounts. Implement rate limiting, account lockout, and anomalous access alerting on login endpoints.
5. ****MEDIUM PRIORITY (Week 3–4):**** Conduct a comprehensive code review of the entire WebGoat codebase (274 files scanned) to identify and remediate any remaining SQL injection patterns beyond the three confirmed locations. Use `semgrep` or equivalent SAST tool in CI/CD to prevent future SQL injection vulnerabilities.
6. ****ONGOING:**** Establish secure SDLC practices: require parameterized queries by default, enable static code analysis (`semgrep`) in pre-commit hooks, perform dynamic security testing (`sqlmap`, OWASP ZAP) on all endpoints before production deployment, and conduct threat modeling for identity and data-access layers.

Methodology & scope

Automated tools — dynamic SQL-injection testing (sqlmap), static code analysis (semgrep + AI), and external attack-surface enumeration — against authorised lab targets. Tool output was consolidated and prioritized by a language model and reviewed by a human analyst before release.